

Can you suggest good conferences or workshops for

Yes — and actually this paper is a **very good seed** for an MSc thesis.

The reason is that the current paper already has the hardest "first step" done:

- it identifies a **clear and non-trivial problem**,
- it proposes a **conceptual framework**,
- it has a **working prototype/instrumentation setup**,
- and it already exposes some **limitations that naturally turn into thesis research questions**.

So the thesis should **not** just be "a bigger version of the paper."

A good MSc thesis should turn this into a **broader, deeper, and more computer-science-shaped contribution** — ideally combining:

1. a system/tooling contribution,
2. a stronger empirical evaluation, and
3. a clear methodological or algorithmic insight.

My main recommendation

Best thesis direction:

Auditing, Explaining, and Optimizing Automatically Constructed Context in LLM-Based Agents

This gives you a thesis that is stronger than the current paper because it moves from:

- “we can observe automatically constructed context”

to

- “we can systematically analyze it, explain it, and improve it.”

That is a much more complete MSc thesis story.

Why this is a strong MSc thesis topic

Your current paper is already good at showing:

- retrieval inserts context,
- memory restoration changes context,
- tool traces accumulate across calls,
- and these cause measurable request-level differences.

That’s a **solid empirical paper**.

But an MSc thesis in CS usually becomes stronger when it adds at least one of:

- a **generalizable software artifact**,
- a **method for detection/classification/optimization**,
- broader evaluation across systems,
- or a user-facing/debugging component.

So the thesis should build on the current work by answering a larger question such as:

How can automatically constructed context in LLM agents be systematically audited, explained, and controlled across frameworks, providers, and workflows?

That is specific enough to be feasible, but broad enough to become a real thesis.

What the thesis should not be

I would avoid making the MSc thesis just:

- “same experiments, but more runs,” or
- “same setup, but with another model,” or
- “same trace analysis with a few more charts.”

Those are useful extensions, but **not enough by themselves** for a strong thesis.

Instead, the thesis should have a stronger **core contribution type**.

Three strong thesis options

Below are the three best directions, in my view.

Option A — Auditing Framework Thesis (my top recommendation)

Thesis idea

Design and evaluate a **general auditing framework** for automatically constructed context in LLM-based agents.

What you would build

A reusable framework/tool that can:

- capture model-visible context at runtime,
- label sources of context (system / retrieval / memory / tool / framework),
- compare context across invocations,
- detect suspicious or undesired patterns,
- and produce trace summaries or visualizations.

Research questions

Examples:

- **RQ1:** How can automatically constructed context be represented in a framework-agnostic way?
- **RQ2:** What types of context construction patterns recur across frameworks, tools, and memory mechanisms?
- **RQ3:** Which auditing signals best identify undesirable behavior such as hidden context growth, irrelevant memory restoration, or privacy-sensitive insertion?
- **RQ4:** How consistent are these patterns across providers and agent frameworks?

Why this is a great MSc thesis

Because it gives you:

- a **software artifact**,
- a **formal or structured representation**,
- experimentation,
- and practical value for developers.

What would make it strong

You would extend the current paper from one controlled LangChain-like setup into something like:

- **multiple frameworks** (e.g., LangChain + another framework or a custom lightweight agent),
- **multiple model providers**,
- **more workflow types**,
- and maybe a **schema or interface standard** for context provenance.

Deliverables

- a tracing/auditing library,
- a context provenance schema,
- an evaluation corpus or benchmark traces,
- and a thesis with case studies.

Difficulty

Moderate and very suitable for MSc.

Option B — Context Optimization Thesis

Thesis idea

Move beyond observing context and ask:

Can we reduce cost, risk, or noise by optimizing automatically constructed context without hurting task performance?

What you would build

A method that:

- detects redundant or low-value context,
- decides what to keep/remove/compress,
- and evaluates the trade-off between:
 - prompt cost,
 - answer quality,
 - tool success,
 - safety/privacy.

Research questions

- **RQ1:** Which automatically added context components are truly useful for downstream task performance?
- **RQ2:** Can context be pruned/compressed while preserving answer quality?
- **RQ3:** How much cost reduction is possible from context-aware optimization?
- **RQ4:** When does removing context harm agent reliability?

Why this is good

This turns the work into a more algorithmic/system design thesis rather than only observational analysis.

Example thesis title

Cost-Aware Optimization of Automatically Constructed Context in LLM-Based Agents

What is needed

You would need evaluation on tasks where performance matters, not just structure:

- question answering,
- task completion,
- tool-use success,
- consistency,
- hallucination/failure cases.

Difficulty

Moderate to high, but very publishable if done well.

Risk

It becomes harder because now you need a good **performance evaluation design**, not only instrumentation.

Option C — Safety / Privacy Thesis

Thesis idea

Focus on the fact that automatically constructed context may include information that the user did not directly type and may not even know is being inserted.

This leads naturally to:

How can unsafe, privacy-sensitive, or policy-violating context construction be detected and mitigated in LLM-based agents?

What you would study

Examples:

- unsafe memory restoration,
- credentials/secrets reappearing in context,
- sensitive tool outputs becoming model-visible,
- excessive cross-turn accumulation,
- invisible prompt growth affecting cost or behavior.

Research questions

- **RQ1:** What categories of privacy/safety risks arise from automatically constructed context?
- **RQ2:** Can runtime traces be used to detect risky context construction before model invocation?
- **RQ3:** What mitigation mechanisms are effective (blocking, redaction, provenance checks, policy filters)?
- **RQ4:** What is the trade-off between safety filtering and answer usefulness?

Why this is attractive

It gives the thesis a very strong “real-world importance” angle.

Difficulty

Moderate, but you need to design a careful evaluation with realistic risk scenarios.

Good if...

You want the thesis to connect to:

- trustworthy AI,
 - secure systems,
 - privacy engineering,
 - agent governance.
-

My honest recommendation: choose Option A with a small slice of B or C

If I were planning this as a real MSc thesis, I would suggest:

Primary thesis

A framework for auditing automatically constructed context in LLM-based agents

and then add **one secondary angle**:

- either **optimization** (cost/noise reduction),
- or **safety/privacy** (risky insertions).

That gives you the best balance between:

- feasibility,
 - originality,
 - engineering contribution,
 - and publishability.
-

A concrete thesis formulation

Here is a version I think would work extremely well.

Proposed thesis title

Auditing and Controlling Automatically Constructed Context in LLM-Based Agents

Core thesis question

How can the runtime-constructed context of LLM-based agents be represented, audited, and controlled across different agent architectures and workflows?

Sub-questions

1. What kinds of context sources appear consistently across agent frameworks?
 2. How can they be traced and labeled in a provider-agnostic way?
 3. What patterns lead to unwanted effects such as hidden cost growth, stale memory use, irrelevant retrieval, or privacy-sensitive exposure?
 4. Can simple control mechanisms reduce these effects without significantly harming utility?
-

What would be “new” beyond the current paper?

The current paper already contributes:

- a schema,
- controlled experiments,
- trace-first analysis,
- token-level consequences.

For the thesis, I would add **three major new elements**.

1) Generalization across systems

Right now the study is intentionally narrow and controlled.

A thesis should broaden that by testing at least some of:

- more than one framework,
- more than one provider/model,
- more than one task family,
- more than one tool type,
- more than one memory mechanism.

This addresses one of the biggest limitations already acknowledged in the paper.

Minimum acceptable version

- 2 frameworks
- 2 models/providers
- 3–4 workflow/task families

That is already a solid MSc-level expansion.

2) A reusable artifact

A thesis becomes much stronger if you produce something other people can use.

Examples:

- a Python tracing library,
- a context provenance schema,
- a visualization dashboard,
- a benchmark suite,
- a dataset of sanitized traces,
- or a policy-checking module.

This makes the thesis feel like **computer science**, not just “more experiments.”

Best artifact idea

A small library like:

- hook into agent framework,
- capture invocation context,
- label context sources,
- compute growth statistics,
- flag risk patterns,
- export machine-readable trace data.

That would be an excellent MSc deliverable.

3) One intervention, not just observation

Observation is good; intervention makes it a stronger thesis.

For example, implement one or more of:

- **context growth alerts,**
- **memory filtering,**
- **retrieval relevance thresholding,**
- **duplicate tool-result suppression,**
- **sensitive content blocking,**
- **context summarization/compression.**

Then evaluate whether the intervention helps.

That creates a full “problem → diagnosis → solution → evaluation” thesis arc.

A very practical thesis structure

Here is a thesis structure that would work well.

Chapter 1 — Introduction

- Problem: model-visible context is runtime-constructed and not fully

transparent.

- Why it matters:
 - reliability,
 - debugging,
 - cost,
 - safety/privacy.
- Thesis goal and research questions.

Chapter 2 — Background

- LLM agents,
- retrieval,
- memory,
- tool use,
- observability,
- provenance,
- prompt/context management.

Chapter 3 — Conceptual model

- Extend your current context schema.
- Define context provenance model more formally.
- Define what kinds of undesirable patterns you care about.

Chapter 4 — Auditing framework design

- Architecture,
- instrumentation interface,
- trace representation,
- labeling strategy,
- metrics,
- implementation details.

Chapter 5 — Experimental setup

- frameworks,
- models,
- tasks,
- memory settings,
- tools,
- datasets / synthetic and maybe real tasks.

Chapter 6 — Empirical analysis

- what context sources appear,
- how trajectories evolve,
- comparative patterns across systems,
- token/cost growth,
- behavioral effects.

Chapter 7 — Control / mitigation mechanism

- your optimization or safety method,
- evaluation of intervention.

Chapter 8 — Discussion

- implications for developers,
- limitations,
- external validity,
- recommendations.

Chapter 9 — Conclusion

That is a very coherent MSc thesis.

Specific research directions that are

especially promising

Here are some concrete thesis questions that could become very good studies.

Direction 1 — Framework-agnostic provenance model

Possible question

Can we define a unified provenance representation for runtime-constructed context across multiple agent frameworks?

Why it is good

It has a strong CS flavor:

- abstraction,
- software architecture,
- evaluation across implementations.

Likely contribution

A schema + library + empirical comparison.

Direction 2 — Context bloat and cost growth

Possible question

How and when do agent workflows accumulate unnecessary prompt context over multi-step execution?

Why it is good

It is very practical and measurable.

Likely contribution

- trace analysis,
 - growth metrics,
 - mitigation strategy,
 - cost/performance trade-off.
-

Direction 3 — Risk-aware memory restoration

Possible question

How can memory restoration be filtered to avoid privacy-sensitive or stale context entering model-visible input?

Why it is good

Very relevant and easy to motivate.

Likely contribution

- risk taxonomy,
 - filtering mechanism,
 - evaluation on memory scenarios.
-

Direction 4 — User-facing explainability

Possible question

How can users and developers be shown what context an agent actually used?

Why it is good

Suitable if you want a human-computer interaction angle.

Likely contribution

- visualization UI,
- explanation interface,
- user study or developer study.

Note

This is very attractive, but may broaden the thesis a lot if you add a full user study.

What data/tasks should you use?

Your current paper uses **synthetic policy facts**, which is great for control.

For the thesis, I would recommend:

Keep synthetic tasks

They are useful because they let you isolate mechanisms cleanly.

Add one more realistic layer

For example:

- public documentation QA,
- API help tasks,
- bug-triage style tasks,
- software engineering assistant tasks,
- or public benchmark-like multi-step tasks.

That way the thesis has both:

- **controlled internal validity**, and
- **better external validity**.

This balance is often ideal for an MSc thesis.

How much scope is appropriate for an MSc?

A good thesis needs a scope that is ambitious but not impossible.

Good scope

- 2 frameworks
- 2 models/providers
- 3 experiment families
- 1 reusable tool/artifact
- 1 intervention method

That is **plenty** for an MSc.

Too much scope

- 5+ frameworks,
- a full benchmark suite,
- multiple user studies,
- heavy theoretical modeling,
- and several mitigation algorithms.

That would likely become too large.

A realistic thesis plan (6–8 months)

Here is a sensible timeline.

Month 1

- refine research questions,

- review related work,
- define thesis scope,
- refactor current codebase.

Month 2

- design generalized trace schema,
- build framework-independent instrumentation interface,
- prepare datasets/tasks.

Month 3

- implement tracing in framework 1 and framework 2,
- run pilot experiments,
- validate logs and metrics.

Month 4

- perform large-scale controlled experiments,
- analyze cross-framework/provider patterns.

Month 5

- implement intervention:
 - optimizer, filter, or risk detector,
- run evaluation.

Month 6

- write main chapters,
- create figures,
- prepare reproducibility artifact.

Month 7–8 (if available)

- polish analysis,

- add a small extra study,
 - prepare workshop/conference paper from thesis.
-

What would make the thesis strong enough to impress examiners?

If I were evaluating it, the strongest signals would be:

1. A clear and original problem

You already have that.

2. A real software contribution

Build an auditing/provenance tool.

3. A careful experimental design

Keep the controlled style from the current paper.

4. Evidence of generalization

At least some breadth beyond one framework/provider.

5. Reproducibility

Sanitized traces, code, schemas, scripts, evaluation data.

6. A practical takeaway

Not only "here is what happens," but also:

- "here is how to detect it,"
 - "here is how to reason about it,"
 - and ideally "here is how to improve it."
-

If you want the thesis to be more publishable

If your goal is not only to graduate but also to line it up for a good paper, then the best combination is:

Thesis version with highest publication potential

- auditing framework
- cross-framework/provider evaluation
- one mitigation method
- artifact release

That combination could produce:

- one thesis,
 - one workshop paper,
 - and potentially one stronger conference submission later.
-

My suggested “best” final thesis concept

If I had to pick one exact thesis direction for you, it would be this:

Thesis proposal

Auditing and Mitigating Automatically Constructed Context in LLM-Based Agents

Main contributions

1. A framework-agnostic provenance model for runtime-constructed context.
2. An auditing toolkit that captures, labels, and summarizes context sources.

3. **A comparative empirical study** across multiple frameworks/models/tasks.
4. **A mitigation component** that detects or reduces undesirable context growth or risky insertion.

Why this is the best choice

Because it is:

- technically strong,
 - clearly novel,
 - feasible for MSc,
 - directly connected to your current paper,
 - and easy to motivate academically and practically.
-

If you want a narrower and safer MSc version

If you want something more contained and less risky, do this:

Safer thesis topic

A Runtime Auditing Framework for Automatically Constructed Context in LLM-Based Agents

In this version:

- skip the optimization/mitigation part if time is tight,
- focus on generalization and tooling,
- and make the deliverable a solid framework + evaluation.

This is probably the **lowest-risk path** to a very good MSc thesis.

If you want a more ambitious /

"clever" thesis

If you want something a bit more cutting-edge:

More ambitious thesis topic

Cost-Aware and Risk-Aware Control of Automatically Constructed Context in LLM Agents

This is stronger conceptually, but riskier because you need:

- good evaluation metrics,
- convincing interventions,
- and stronger behavioral analysis.

I would only choose this if you are confident you can invest more time.

My bottom-line advice

Best path

Treat the current paper as:

- **pilot study + foundation chapter + initial publication seed**

and make the thesis about:

- **building a reusable auditing system,**
- **broadening the empirical analysis, and**
- **adding one practical control mechanism.**

That gives you the strongest MSc story.

If you want, I can do one of these next:

1. **write a concrete MSc thesis proposal** (title, abstract, RQs, methodology, contributions), or

2. **suggest 3 thesis topics ranked by difficulty and publishability, or**
3. **draft a chapter-by-chapter thesis outline based directly on your current paper.**